

CO2 – AT2 – Debugging Task

Question 1

A K-Means clustering algorithm produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-Means++, normalization, and silhouette analysis. (5 Marks)

Theory

Problem

K-Means is an unsupervised machine learning algorithm used to group similar data points into clusters. Sometimes it produces different cluster assignments even for similar datasets because of random centroid initialization and other factors.

Possible Reasons

- Poor initialization of centroids.
- Incorrect selection of the number of clusters (K).
- Presence of outliers.
- Different feature scales.
- Convergence to a local optimum.

Debugging Approaches

- Run K-Means multiple times.
- Visualize the clusters.
- Normalize the dataset.
- Remove outliers.
- Compare different values of K.

Optimization Strategies

- Use **K-Means++** initialization.
- Apply **StandardScaler** for normalization.
- Select the best K using **Silhouette Score**.
- Increase the number of iterations if necessary.

Conclusion

Using K-Means++, feature normalization, and silhouette analysis improves clustering accuracy and produces consistent results.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Load dataset
iris = load_iris()
X = iris.data

# Normalize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply K-Means++
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)

labels = kmeans.fit_predict(X_scaled)

print("Cluster Labels:")
print(labels)

print("\nCluster Centers:")
print(kmeans.cluster_centers_)

score = silhouette_score(X_scaled, labels)
```

```
print("\nSilhouette Score:", round(score,3))
```

Sample Output

Cluster Labels:

```
[1 1 1 1 1 ...]
```

Cluster Centers:

```
[[...]
```

```
[...]
```

```
[...]]
```

Silhouette Score: 0.46

Question 2

A Support Vector Machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

Theory

Problem

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression. Poor performance on nonlinear data usually occurs because of an inappropriate kernel or improper parameter settings.

Possible Reasons

- Wrong kernel selection.
- Improper values of C and Gamma.
- Features are not scaled.
- Noisy or irrelevant features.
- Overfitting or underfitting.

Debugging Techniques

- Try different kernels.

- Scale input features.
- Perform cross-validation.
- Analyze the confusion matrix.
- Evaluate accuracy and F1-score.

Optimization Methods

- Use the **RBF kernel** for nonlinear datasets.
- Tune **C** and **Gamma** using Grid Search.
- Normalize features using StandardScaler.
- Remove unnecessary features.
- Perform cross-validation.

Conclusion

Selecting the correct kernel, scaling the data, and tuning hyperparameters significantly improve SVM accuracy.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.3, random_state=42)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# SVM Model
model = SVC(kernel='rbf', C=1.0, gamma='scale')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

Sample Output

Accuracy: 1.0

Question 3

A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies. (5 Marks)

Theory

Problem

Recurrent Neural Networks (RNNs) process sequential data. However, they struggle to remember information from earlier time steps, making it difficult to learn long-term dependencies.

Possible Causes

- Vanishing gradients.
- Exploding gradients.
- Limited memory capacity.
- Poor weight initialization.
- Very long input sequences.

Debugging Steps

- Monitor training and validation loss.
- Observe gradient values.
- Experiment with learning rates.
- Check preprocessing and sequence length.
- Compare RNN with LSTM or GRU.

Optimization Strategies

- Replace RNN with **LSTM** or **GRU**.
- Use **Attention Mechanisms**.
- Apply **Gradient Clipping**.
- Use pretrained word embeddings.
- Train with more data and apply dropout.

Conclusion

LSTM and GRU networks effectively solve the long-term dependency problem and improve NLP model performance.

Python Program

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, LSTM, Dense

from tensorflow.keras.preprocessing.sequence import pad_sequences


# Sample sequences

X = [
```

```
[1,2,3,4],  
[2,3,4],  
[1,3,4,5],  
[2,5]  
]
```

```
# Labels
```

```
y = np.array([1,0,1,0])
```

```
# Padding
```

```
X = pad_sequences(X, maxlen=5)
```

```
# Build Model
```

```
model = Sequential()
```

```
model.add(Embedding(input_dim=10, output_dim=8, input_length=5))
```

```
model.add(LSTM(16))
```

```
model.add(Dense(1, activation='sigmoid'))
```

```
model.compile(optimizer='adam',  
              loss='binary_crossentropy',  
              metrics=['accuracy'])
```

```
# Train
```

```
model.fit(X, y, epochs=10)
```

```
# Prediction
```

```
pred = model.predict(X)
```

```
print("Predictions:")
```

```
print(pred)
```

Sample Output

Epoch 1/10

...

Epoch 10/10

Predictions:

[[0.91]

[0.18]

[0.87]

[0.15]]